Understanding Memory in C

A technical paper by Kent Tonino

Preface

In C, you have direct control over memory. In Python or Go, memory is managed for you, variables appear when you need them and dissappear when you don't, without you thinking about it. In C, you are responsible. You decide where data lives, how long it lasts, and when to release it.

For me, that responsibility is not a burden, it is a capability. Understanding memory in C helps you understand what is going on under the hood. With this out of curiosity paper, it covers memory from the ground up, on how a C program's memory is organised, how the stack and heap work, what pointers actually are, why structs have unexpected sizes, what goes wrong when memory is misused, and how to find those bugs.

01. Memory

Before anything else, let's establish what memory actually is.

From your program's perspective, memory is just a very long sequence of bytes, each with a unique number called **address**. On a 64-bit system, addresses are 64-bit numbers, giving a a theoretical address space of 2^{64} bytes. On a 32-bit system, addresses are 32-bit numbers.

Think of memory as a very long street of numbered houses. Each house holds exactly one byte. The house number is the address. When your program stores a value, it's putting data into one or more of those houses. When it reads a value, it's looking up a house by number and reading what's inside.

```c
int x = 42;
// prints 42
printf("value: %d\n", x);
// prints something like 0x7ffd3a2c1b4c
printf("address: %p\n", (void*)&x);
```

The `&` operator gives you the address (house number) of a variable. On most systems, an `int` occupies 4 consecutive bytes, so `x` 4 houses starting at that address.